

Audio

Ren'Py supports playing music and sound effects in the background, using the following audio file formats:

- Opus
- Ogg Vorbis
- MP3
- MP2
- FLAC
- WAV (uncompressed 16-bit signed PCM only)

On the web browser, Ren'Py will check a list of audio formats, and enable a mode that is faster and less prone to skipping if the web browser supports all modes on the list. If your game is using only mp3s, and skips on Safari, then consider changing

```
config.webaudio_required_types .
```

Ren'Py supports an arbitrary number of audio channels. There are two normal channels defined by default:

- `music` - A channel for music playback.
- `sound` - A channel for sound effects.

Normal channels support playing and queueing audio, but only play back one audio file at a time. New normal channels can be registered with `renpy.music.register_channel()`.

The Music Volume, Sound Volume, and Voice Volume settings of the in-game preferences menu are used to set individual volumes for these channels. See [Volume](#) for more information.

In addition to the normal channels, there are two special channels:

- `audio` - **The audio channel supports playing back multiple audio files at one time,**
but does not support queueing sound or stopping playback.
- `voice` - The voice channel is used by Ren'Py's voice system. Ren'Py will automatically play and stop voice files as characters speak, and may adjust the volume of the voice channel.

Sounds can also be set to play when buttons, menu choices, or imagemaps enter their hovered or activated states. See [Button Style Properties](#). Two configuration variables, `config.main_menu_music` and `config.game_menu_music` allow for the given music files to be played as the main and game menu music, respectively.

In-game, the usual way to play music and sound in Ren'Py is using the three music/sound statements. Audio files can either be provided directly as strings, or as defined names within the audio namespace.

Audio Directory and Namespace

When Ren'Py searches for an audio file used by audio statements or functions, it will start inside the `game` directory. If the provided file is not found there, it will also look in the `game/audio` directory. For example:

```
play music "opening.ogg"
```

will first look for `game/opening.ogg`. If not found, Ren'Py will look for `game/audio/opening.ogg`. This is consistent with audio files in further subdirectories. The following statement:

```
play music "my_music/opening.ogg"
```

will first look for `game/my_music/opening.ogg`, before looking for `game/audio/my_music/opening.ogg` if the former is not found.

Files found in the `game/audio` directory, **as well as its subdirectories**, that end with a supported extension (currently, .wav, .mp2, .mp3, .ogg, and .opus) are also automatically placed by Ren'Py into the audio namespace. This means that `game/audio/Town_theme.ogg` can be played with:

```
play music town_theme
```

The usable name is determined by stripping the extension and forcing the rest of the filename to lower case. This is only the case for audio files with names that can be expressed as Python variables, for example: `my song.mp3`, `8track.opus`, or `this-is-a-song.ogg` will not work. Additionally, if two or more files would end up defined under the same name, only the first file (determined alphabetically by its path and filename, extension included) will be defined.

When a file isn't placed automatically into the audio namespace, either due to an incompatible name or being inside a different directory, it can still be placed there manually with the `define` statement.

For example, one can write:

```
define audio.sunflower = "my_music/sun-flower-slow-jam.ogg"
```

and then use:

```
play music sunflower
```

The `play` and `queue` statements always evaluate their arguments in the audio namespace. Functions do not, meaning they will not work with `sunflower`, but will work with `audio.sunflower` instead.

Play Statement

The `play` statement is the most common way used to play sound and music. If a file is currently playing on a normal channel, it is interrupted and replaced with the new file.

The name of a channel is expected following the keyword `play`. (Usually, this is either "sound", "music", "voice", or "audio"). This is followed by audiofile(s), where audiofile(s) can be one file or list of files. When the list is given, the item of it is played in order.

The `fadein` and `fadeout` clauses are optional. Fadeout gives the fadeout time for currently playing music, in seconds, while fadein gives the time it takes to fade in the new music. If fadeout is not given, `config.fadeout_audio` is used.

The `loop` and `noloop` clauses are also optional. The loop clause causes the music to loop, while noloop causes it to play only once. If neither of them are given, the default of the channel is used.

```
play music "mozart.ogg"
play sound "woof.mp3"
play myChannel "punch.wav" # 'myChannel' needs to be defined with
renpy.music.register_channel().
```

```
"We can also play a list of sounds, or music."
play music [ "a.ogg", "b.ogg" ] fadeout 1.0 fadein 1.0
```

When the `if_changed` clause is provided, and if the given track is currently playing on the channel, the play instruction doesn't interrupt it.

```
label market_side:
    play music market
    "We're entering the market."
    jump market_main

label market_main:
    play music market if_changed
    "Maybe we just entered the market, maybe we were already there."
    "If we were already there, the music didn't stop and start over, it just continued."
    jump market_main
```

The `volume` clause is also optional, and specifies a relative amplitude for the track, between 0.0 and 1.0. This makes it possible to adjust the amplitude a track is played at, each time it's played.

```
play sound "woof.mp3" volume 0.5
```

On the audio channel, multiple play statements play multiple sounds at the same time:

```
play audio "sfx1.opus"
play audio "sfx2.opus"
```

A variable may be used instead of a string here. If a variable exists in the [audio namespace](#), it's used in preference to the default namespace:

```
play music illurock
```

Files placed into the audio namespace may automatically define variables that can be used like this.

Ren'Py supports a feature that can ensure that audio files start playing at the same time. This feature is enabled on looping audio channels (like music) by default, but can also be enabled by the `synchro_start` option to `renpy.music.register_channel()` or `renpy.music.play()`.

When synchro start is enabled and multiple play statements are run at the same time, the audio in each channel will start synchronized. Specifically, the audio will start:

- When the audio files on every channel have been loaded and audio samples are available.

- When all all channels have been faded out.

New audio will start playing when both conditions are met.

Stop Statement

The `stop` statement begins with the keyword `stop`, followed by the name of a channel to stop sound on. It may optionally have a `fadeout` clause. If the fadeout clause is not given, `config.fadeout_audio` is used.

```
stop sound
stop music fadeout 1.0
```

Queue Statement

The `queue` statement is used to queue up audio files. They will be played when the channel finishes playing the currently playing file.

The queue statement begins with keyword `queue`, followed by the name of a channel to play sound on. It optionally takes the `fadein`, `loop` and `no loop` clauses.

```
queue sound "woof.mp3"
queue music [ "a.ogg", "b.ogg" ]
```

Queue also takes the `volume` clause.

```
play sound "woof.mp3" volume 0.25
queue sound "woof.mp3" volume 0.5
queue sound "woof.mp3" volume 0.75
queue sound "woof.mp3" volume 1.0
```

When multiple queue statements are given without an interaction between them, all sound files are added to the queue. After an interaction has occurred, the first queue statement clears the queue, unless it has already been cleared by a play or stop statement.

A variable may be used instead of a string here. If a variable exists in the [audio namespace](#), it's used in preference to the default namespace:

```
define audio.woof = "woof.mp3"

# ...

play sound woof
```

The advantage of using these statements is that your program will be checked for missing sound and music files when lint is run. The functions below exist to allow access to allow music and sound to be controlled from Python, and to expose advanced (rarely used) features.

Partial Playback

Ren'Py supports partial playback of audio files. This is done by putting a playback specification, enclosed in angle brackets, at the start of the file. The partial playback specification should consist of alternating property name and value pairs, with every thing separated by spaces.

The values are always interpreted as seconds from the start of the file. The three properties are:

from

Specifies the position in the file at which the first play-through begins playing. (This defaults to 0.0 seconds.)

to

Specifies the position in the file at which the file ends playing. (This defaults to the full duration of the file.)

loop

Specifies the position in the file at which the second and later play-throughs begin playing. (This defaults to the start time given by **from** if specified, or to the start of the file.)

For example:

```
play music "<from 5 to 15.5>waves.opus"
```

will play 10.5 seconds of waves.opus, starting at the 5 second mark. The statement:

```
play music "<loop 6.333>song.opus"
```

will play song.opus all the way through once, then loop back to the 6.333 second mark before playing it again all the way through to the end.

Sync Start Position

The position in the file at which the clip begins playing can also be synced to another channel with a currently-playing track using a filename like "<sync channelname>track.opus", where channelname is the name of the channel, which could be "music", "sound", or any other registered channels.

This can be used to sync multi-layered looping tracks together. For example:

```
play music_2 [ "<sync music_1>layer_2.opus", "layer_2.opus" ]
```

Will play `layer_2.opus` with the start time synced to the current track in channel music_1 in the first iteration, before playing the whole track in subsequent iterations. (By default, the `layer_2.opus` start time will remain modified even in subsequent iterations in the loop.)

Volume

The volume at which a given track is going to be played depends on a number of variables:

- the "main" mixer's volume
- the volume of the mixer which the channel relates to
- the volume of the channel
- the relative amplitude of the track itself
- the relative amplitude associated with the filename

These four volumes are values between 0 and 1, and their multiplication results in the volume the track will be played at.

For example, if the main volume is 80% (or 0.8), the mixer's volume is 100%, the channel volume is 50% (0.5) and the track's relative volume is 25% (0.25), and the filename's relative volume is 50% (0.5), the resulting volume is $.8 * 1 * .5 * .25 * .5 = .05$ so 5%.

Note that while all of these volumes are amplitudes, the mixers are presented as decibels in the preferences menu.

The mixers' volumes can be set using `preferences.set_mixer()`, using the `SetMixer()` action, or using the `Preference()` action with the `"mixer <mixer> volume"` key. The "audio" and "sound" channels relate to the "sfx" mixer, the "music" channel to the "music" mixer and the "voice"

channel to the "voice" mixer. Every channel additionally relates to the "main" mixer, as shown above.

A channel's volume can be set using `renpy.music.set_volume()`. It is only useful when several channels use the same mixer. The `mixer` parameter of the `renpy.music.register_channel()` function sets to which mixer the registered channel relates, creating it in the process if it doesn't already exist.

A track's relative volume is set with the `volume` clause of the [Play Statement](#).

A filename's relative volume is set with a volume clause in angle brackets, such as "<volume 0.5>track.opus".

In addition to these volume values, there is the mute flag of the mixer which the channel relates to. If enabled, it will reduce the played volume to 0. They can be set using the `SetMute()` or `ToggleMute()` actions, or using the `Preference()` action with the "mixer <mixer> mute" key, or using the `preferences.set_mute()` function.

Playing Silence

A specified duration of silence can be played using a filename like "<silence 3.0>", where 3.0 is the number of seconds of silence that is desired. This can be used to delay the start of a sound file. For example:

```
play audio [ "<silence .5>", "boom.opus" ]
```

Will play silence for half a second, and then an explosion sound.

Actions

See [Audio Actions](#).

Functions

`class` `AudioData`(*data*, *filename*)

This class wraps a bytes object containing audio or video data, so it can be passed to the audio/video playback system. The audio or video data should be contained in some format Ren'Py supports. (For examples RIFF WAV format headers, not unadorned samples.)

Despite the name, this can be used to represent video data as well as audio data.

data

A bytes object containing the audio file data.

filename

A synthetic filename associated with this data. It can be used to suggest the format *data* is in, and is reported as part of error messages.

If this starts with angle bracket, it can supply properties to the audio, like from and to times.

Once created, this can be used wherever an audio filename is allowed. For example:

```
define audio.easteregg = AudioData(b'...', 'sample.wav')  
play sound easteregg
```

renpy.mark_audio_seen(*filename*)

Marks the given filename as if it has been already played on the current user's system.

renpy.mark_audio_unseen(*filename*) 🔒

Marks the given filename as if it has not been played on the current user's system yet.

renpy.play(*filename*, *channel*=None, **kwargs**)**

Plays a sound effect. If *channel* is None, it defaults to `config.play_channel`. This is used to play sounds defined in styles, `hover_sound` and `activate_sound`.

renpy.seen_audio(*filename*)

Returns True if the given filename has been played at least once on the current user's system.

renpy.music.channel_defined(*channel*)

Returns True if the channel exists, or False otherwise.

renpy.music.get_all_mixers()

This gets all mixers in use.

renpy.music.get_duration(*channel*='music')

Returns the duration of the audio or video file on *channel*. Returns 0.0 if no file is playing on *channel*, or the duration is unknown. Some formats - notably MP3 - do not include duration information in a format Ren'Py can access.

renpy.music.get_loop(channel='music')

Return a list of filenames that are being looped on *channel*, or None if no files are being looped. In the case where a loop is queued, but is not yet playing, the loop is returned, not the currently playing music.

renpy.music.get_pause(channel='music')

Returns the pause flag for *channel*.

renpy.music.get_playing(channel='music')

If the given channel is playing, returns the playing file name. Otherwise, returns None.

renpy.music.get_pos(channel='music')

Returns the current position of the audio or video file on *channel*, in seconds. Returns None if no audio is playing on *channel*.

As this may return None before a channel starts playing, or if the audio channel involved has been muted, callers of this function should always handle a None value.

renpy.music.is_playing(channel='music')

Returns True if the channel is currently playing a sound, False if it is not, or if the sound system isn't working.

renpy.music.play(filenames, channel='music', loop=None, fadeout=None, synchro_start=None, fadein=0, tight=None, if_changed=False, relative_volume=1.0)

This stops the music currently playing on the numbered channel, dequeues any queued music, and begins playing the specified file or files.

filenames

This may be a single file, or a list of files to be played.

channel

The channel to play the sound on.

loop

If this is True, the tracks will loop while they are the last thing in the queue.

fadeout

If not None, this is a time in seconds to fade for. Otherwise the fadeout time is taken from config.fadeout_audio. This is ignored if the channel is paused when the music is played.

synchro_start

When True, all channels that have `synchro_start` set to true will start playing at exactly the same time. This may lead to a pause before the channels start playing. This is useful when playing two audio files that are meant to be synchronized with each other.

If None, this takes its value from the channel.

fadein

This is the number of seconds to fade the music in for, on the first loop only.

tight

If this is True, then fadeouts will span into the next-queued sound. If None, this is true when loop is True, and false otherwise.

if_changed

If this is True, and the music file is currently playing, then it will not be stopped/faded out and faded back in again, but instead will be kept playing. (This will always queue up an additional loop of the music.)

relative_volume

This is the volume relative to the current channel volume. The specified file will be played at that relative volume. If not specified, it will always default to 1.0, which plays the file at the original volume as determined by the mixer, channel and secondary volume.

This clears the pause flag for *channel*.

`renpy.music.pump()`

This 'pumps' the audio system. Normally, the effects of the `play`, `queue`, and `stop` statements and the function equivalents take place at the start of the next interaction. In some cases, the effects of multiple statements can cancel each other out - for example, a play followed by a stop causes the track to never be played.

If this function is called between the play and stop, the track will begin playing before this function returns, which then allows the track to be faded out.

```
play music "mytrack.opus"  
$ renpy.music.pump()  
stop music fadeout 4
```

`renpy.music.queue(filenames, channel='music', loop=None, clear_queue=True, fadein=0, tight=None, relative_volume=1.0)`

This queues the given filenames on the specified channel.

filenames

This may be a single file, or a list of files to be played.

channel

The channel to play the sound on.

loop

If this is True, the tracks will loop while they are the last thing in the queue.

clear_queue

If True, then the queue is cleared, making these files the files that are played when the currently playing file finishes. If it is False, then these files are placed at the back of the queue. In either case, if no music is playing these files begin playing immediately.

fadein

This is the number of seconds to fade the music in for, on the first loop only.

tight

If this is True, then fadeouts will span into the next-queued sound. If None, this is true when loop is True, and false otherwise.

relative_volume

This is the volume relative to the current channel volume. The specified file will be played at that relative volume. If not specified, it will always default to 1.0, which plays the file at the original volume as determined by the mixer, channel and secondary volume.

This clears the pause flag for *channel*.

```
renpy.music.register_channel(name, mixer, loop=None, stop_on_mute=True, tight=False,  
file_prefix="", file_suffix="", buffer_queue=True, movie=False, framedrop=True)
```

This registers a new audio channel named *name*. Audio can then be played on the channel by supplying the channel name to the play or queue statements.

name

The name of the channel. It should not contain spaces, as this is reserved for Ren'Py's internal use, and should be a [valid identifier](#) for the syntax of the [Play Statement](#) to be usable.

mixer

The name of the mixer the channel uses. By default, Ren'Py knows about the "music", "sfx", and "voice" mixers. Using other names is possible, and will create the mixer if it doesn't already exist, but making new mixers reachable by the player requires changing the preferences screens.

loop

If True or None, sounds on this channel loop by default. If False, sounds on this channel do not loop by default.

stop_on_mute

If true, music on the channel is stopped when the channel is muted.

tight

If true, sounds will loop even when fadeout is occurring. This should be set to True for a sound effects or seamless music channel, and False if the music fades out on its own.

file_prefix

A prefix that is prepended to the filenames of the sound files being played on this channel.

file_suffix

A suffix that is appended to the filenames of the sound files being played on this channel.

buffer_queue

Should we buffer the first second or so of a queued file? This should be True for audio, and False for movie playback.

movie

If true, this channel will be set up to play back videos.

framedrop

This controls what a video does when lagging. If true, frames will be dropped to keep up with realtime and the soundtrack. If false, Ren'Py will display frames late rather than dropping them.

synchro_start

Does this channel participate in synchro start? Synchro start determines if the channel will start playing at the same time as other channels. If None, this defaults to *loop* if *movie* is False, and False otherwise.

`renpy.music.set_audio_filter(channel, audio_filter, replace=False, duration=0.016)`

Sets the audio filter for sounds about to be queued to *audio_filter*.

audio_filter

Must be a an [audio filter](#) or list of audio filters, or None to remove the audio filter.

replace

If True, the audio filter replaces the current audio filter immediately, changing currently playing and queued sounds. If False, the audio filter will be used the next time a sound is played or queued.

duration

The duration to change from the current to the new filter, in seconds. This prevents a popping sound when changing filters.

renpy.music.set_mixer(*channel*, *mixer*, *default=False*)

This sets the name of the mixer associated with a given channel. By default, there are two mixers, 'sfx' and 'music'. 'sfx' is on channels 0 to 3, and 'music' on 3 to 7. The voice module calls this function to set channel 2 to voice. You can create your own mixer, but will need to add a preference if you wish to allow the user to set it.

This function should only be called in an init block.

renpy.music.set_pan(*pan*, *delay*, *channel='music'*)

Sets the pan of this channel.

pan

A number between -1 and 1 that control the placement of the audio. If this is -1, then all audio is sent to the left channel. If it's 0, then the two channels are equally balanced. If it's 1, then all audio is sent to the right ear.

delay

The amount of time it takes for the panning to occur.

channel

The channel the panning takes place on, defaulting to the music channel.

renpy.music.set_pause(*value*, *channel='music'*)

Sets the pause flag for *channel* to *value*. If True, the channel will pause, otherwise it will play normally.

renpy.music.set_queue_empty_callback(*callback*, *channel='music'*)

This sets a callback that is called when the queue is empty. This callback is called when the queue first becomes empty, and at least once per interaction while the queue is empty.

The callback is called with no parameters. It can queue sounds by calling `renpy.music.queue` with the appropriate arguments. Please note that the callback may be called while a sound is playing, as long as a queue slot is empty.

renpy.music.set_volume(*volume*, *delay=0*, *channel='music'*)

Sets the volume of this channel, as a fraction of the volume of the mixer controlling the channel.

volume

This is a number between 0.0 and 1.0, and is interpreted as a fraction of the mixer volume for the channel.

delay

It takes *delay* seconds to change/fade the volume from the old to the new value. This value is persisted into saves, and participates in rollback.

channel

The channel to be set

renpy.music.stop(*channel='music'*, *fadeout=None*)

This stops the music that is currently playing, and dequeues all queued music. If *fadeout* is *None*, the music is faded out for the time given in `config.fadeout_audio`, otherwise it is faded for *fadeout* seconds.

This sets the last queued file to *None*.

channel

The channel to stop the sound on.

fadeout

If not *None*, this is a time in seconds to fade for. Otherwise the fadeout time is taken from `config.fadeout_audio`. This is ignored if the channel is paused.

Sound Functions

Most `renpy.music` functions have aliases in `renpy.sound`. These functions are similar, except they default to the sound channel rather than the music channel, and default to not looping.